

Applying Design Patterns in a Microservices-Based Digital Wallet Using Spring Boot and FastAPI

Julian David Pulido Carreño cod: 20231020064

Facultad de Ingenieria, Universidad Distrital Francisco Jose de Caldas

	CONTENTS		6	sequence diagram of transfer money	5
			7	Activity diagram of transaction history	5
			8	activity diagram of money transfer	6
			9	Deployment diagram	6
			10	Component diagram	6
I	Introduction	1			
II	Problem Definition	2			
III	Technical and Functional Aspects of the Project	2			
III-A	User stories	2			
III-B	Requirements	2			
	III-B1 Functional Requirements . .	2			
	III-B2 Non-Functional Requirements	3			
III-C	Technical considerations	3			
IV	UML Diagrams	4			
IV-A	Class Diagram	4			
IV-B	Sequence diagram: Login and Money transfer	5			
IV-C	Activity Diagram for transaction history and money transfer	5			
IV-D	Deployment diagram	5			
IV-E	Component diagram	5			
V	SOLID principles in architecture	5			
VI	Analysis and detail of the implementation of design patterns	8			
VII	Analysis of the implementation of good programming practices and anti-patterns.	9			
VIII	Conclusion and Recommendations	9			

LIST OF FIGURES

1	Digital Wallet	2
2	Class Diagram java services	4
3	Class Diagram API Gateway and Python services	4
4	class diagram command and observer pattern . .	4
5	Sequence Diagram of login	5

LIST OF TABLES

I	Evaluation of Design Patterns and Practices in the Project	9
---	--	---

Abstract—This project implements a structured and scalable architecture by integrating various design patterns and best programming practices. By leveraging the **Command Pattern**, **Observer Pattern**, **Repository Pattern**, and **API Gateway Pattern**, the system ensures modularity, maintainability, and adherence to the SOLID principles. Additionally, the use of dependency injection and separation of concerns enhances flexibility and reduces tight coupling between components.

The study evaluates the effectiveness of these patterns in managing transactions, user authentication, and data persistence while minimizing the risks associated with common anti-patterns. The findings highlight that the applied patterns contribute to a robust and extensible system, with a particular emphasis on loose coupling and adherence to software design principles. However, potential concerns such as an overloaded API Gateway and repository-level dependencies are identified as areas for future improvement.

This report provides an in-depth analysis of the architectural decisions made, the advantages gained, and the challenges encountered. The conclusions drawn from this study offer insights into designing flexible and scalable software systems, ensuring that new functionalities can be integrated with minimal disruption to existing structures.

I. INTRODUCTION

Modern software development requires architectures that support **scalability**, **maintainability**, and **adaptability**. Poorly structured systems often lead to increased technical debt, making modifications and extensions costly and error-prone. To address these challenges, this project employs **design patterns** and **best programming practices** to develop a modular and efficient system for managing user authentication, financial transactions, and account operations.

The system is built upon several key design patterns, including the **Command Pattern**, which encapsulates transaction-related operations, the **Observer Pattern**, which decouples event-driven functionalities such as logging

and notifications, and the **Repository Pattern**, which abstracts data access layers. Additionally, an **API Gateway Pattern** is used to centralize communication between microservices, ensuring a streamlined request-handling mechanism. These patterns align with the **SOLID principles**, particularly the Open-Closed Principle and Dependency Inversion Principle, fostering flexibility in future system enhancements.

Beyond design patterns, best programming practices such as **dependency injection**, **separation of concerns**, and **modularity** are integrated into the system. These principles contribute to reduced code complexity and enhanced maintainability, preventing common anti-patterns such as **Spaghetti Code** and **Shotgun Surgery**. However, while these strategies improve the system's architecture, potential risks such as **API Gateway overload** and **tight coupling at the repository level** must be carefully managed.

II. PROBLEM DEFINITION

In today's digital era, financial transactions have become an integral part of everyday life. However, traditional banking systems often involve long processing times, high transaction fees, and limited accessibility for users without bank accounts. Many individuals seek a secure, efficient, and user-friendly solution to manage their funds, make transactions, and track their financial history in real-time.

This project aims to develop a **digital wallet** that provides users with a seamless financial management experience. The wallet should allow users to securely log in, check their account balance, deposit and withdraw funds, transfer money to other accounts, and view their transaction history. Additionally, the system must ensure real-time transaction notifications and status tracking to enhance transparency and security.

The main challenges include ensuring a secure authentication system, maintaining data integrity, enabling fast and reliable transactions, and providing a scalable solution that can handle multiple concurrent users. By addressing these issues, this digital wallet will offer an accessible and modern alternative to traditional banking methods, improving financial inclusion and efficiency for users.

III. TECHNICAL AND FUNCTIONAL ASPECTS OF THE PROJECT

A. User stories

User Stories are a key tool in software development, helping to capture functional requirements from an end-user perspective. They enhance communication, align development with business needs, and support Agile methodologies by breaking down complex requirements into manageable tasks. For this reason, we define these User Stories to capture the functional needs of the digital wallet, ensuring a user-centric development.

- As a user, I want to log in using my number phone and password so that I can access my digital wallet and perform transactions.
- As a user, I want to see my account balance so that I can know how much money I have available before making a transaction.



Fig. 1. Digital Wallet

- As a user, I want to deposit funds into my account so that I can increase my available balance in the wallet.
- As a user, I want to withdraw funds from my account so that I can use my money in cash.
- As a user, I want to see my transaction history so that I can track all the transactions I have made over time.
- As a user, I want to know each transaction in the transaction history so that know where my money went.
- As a user, I want to receive a notification after each transaction so that I can be informed whether it was successful or not.
- As a user, I want to check the status of a transaction so that I can confirm whether it was completed, pending, or failed.
- As a user, I want to transfer money to other accounts so that I can pay my obligations
- As a user, I want to log out of the application so that my session is securely terminated.
- As a user, I want to deposit money in my digital wallet so that I can transfer money from anyplace.
- As a user, I want to know the transactions in an interval of time so that I can found my movements easier.

B. Requirements

1) Functional Requirements

The digital wallet system must meet the following functional requirements:

- The system must allow users to filter transactions within a specific time interval.
- The system must inform users after each transaction, informing them if it was successful.

- The system must allow users to check the real-time status of a transaction (completed, pending, or failed).

2) Non-Functional Requirements

The digital wallet must also adhere to the following non-functional requirements to ensure system efficiency, security, and usability:

- 1) Performance & Scalability
 - The system must respond to user actions (such as balance inquiries and transaction history retrieval) within **2 seconds**.
- 2) Security
 - All user credentials and sensitive data must be securely encrypted.
- 3) Availability & Reliability
 - The system must provide real-time transaction status updates.
- 4) Maintainability & Compatibility
 - The system must allow seamless updates and maintenance without downtime.
 - The system must be compatible with **web browsers**.

C. Technical considerations

In the **digital wallet** project, two **backends** were developed to manage different system modules. One backend handles **user management**, while the other is responsible for **transaction processing** and business logic.

To facilitate communication between these microservices, an **API Gateway** was implemented. Its main role is to hide internal services and enable seamless interaction. For example, during the **login** process, the request is sent to the **API Gateway**, which then redirects it to the **user service controller** (`java_services.controllers.UserController.java`). This controller processes the request by invoking the **user service** (`java_services.services.UserServices.java`), where data validation occurs. Finally, the **user repository** (`java_services.repositories.UserRepositories.java`) verifies the credentials by checking the stored JSON file (`java_services.repositories.data.users.json`).

a) Microservices Implementation

User Service

The **user microservice** is developed in **Java** using the **Spring Boot** framework, which efficiently manages the REST API. Additionally, **Maven** is used for dependency management and packaging. The microservice follows a modular architecture consisting of **controllers, services, and repositories**.

b) Transaction and Account Service

The **transaction and account microservice** is implemented in **Python** using **FastAPI**, a framework that efficiently handles web requests. Like the user microservice, it follows a structured architecture based on **controllers, services, and repositories**.

c) Transaction Process (Deposit Example)

To access the transaction service functionality, the client sends a request to the **API Gateway**, which redirects it to the corresponding microservice. A typical example is the **deposit** function:

- 1) The authenticated user provides the deposit amount to the **API Gateway**.
- 2) The **API Gateway** forwards the request to the **transaction controller** (`Python_Services.controllers.transfer.py`).
- 3) The **controller** passes the request to the **service layer** (`Python_Services.services.transfer.py`), which validates the data and invokes the **deposit command**.
- 4) The **deposit command** interacts with the **transaction repository** (`Python_Services.repositories.transfer.py`), which updates the account balance in the JSON file (`Python_Services.repositories.data.accounts`).

d) Technologies and Tools

The project incorporates a variety of technologies and tools to optimize performance, scalability, and maintainability:

Backend Development

- **Spring Boot**: Used for developing the user microservice, enabling rapid development of REST APIs and dependency injection.
- **Maven**: Manages dependencies and automates the build process for the Java microservice.
- **FastAPI**: A high-performance web framework for the Python microservice, allowing asynchronous processing and automatic API documentation.
- **Uvicorn**: An ASGI server used to run the FastAPI application efficiently.

Data Storage and Persistence

- **JSON Files**: Used for data persistence, ensuring a lightweight and structured way to store user and transaction data.
- **PostgreSQL (Future Upgrade)**: A potential database upgrade for relational data management with improved consistency and scalability.
- **MongoDB (Future Upgrade)**: A NoSQL alternative for handling large volumes of unstructured or semi-structured data.

Communication and Integration

- **API Gateway**: Acts as a single entry point, routing requests to the appropriate microservices and enhancing security by hiding internal services.

- **RESTful API:** Ensures standardized and efficient communication between the microservices.
- **Message Queues (Future Upgrade):** Implementing RabbitMQ or Kafka could enable asynchronous message processing, improving system performance.

Containerization and Deployment

- **Docker:** Used to containerize the microservices, providing an isolated and consistent runtime environment across different platforms.
- **Docker Compose:** Automates the deployment of multiple services and simplifies dependency management.
- **Kubernetes (Future Upgrade):** A possible enhancement for automating deployment, scaling, and management of containerized applications.

Security and Configuration Management

- **python-dotenv:** Manages environment variables securely, improving configuration flexibility.
- **JWT (Future Upgrade):** A planned implementation for secure user authentication and authorization.

The **Command Pattern**, implemented for transaction management, is further detailed in the **design pattern analysis and implementation** section.

IV. UML DIAGRAMS

The class diagram of the project (complete in repository: <https://github.com/modelos1/Proyecto1/tree/main/Files>)

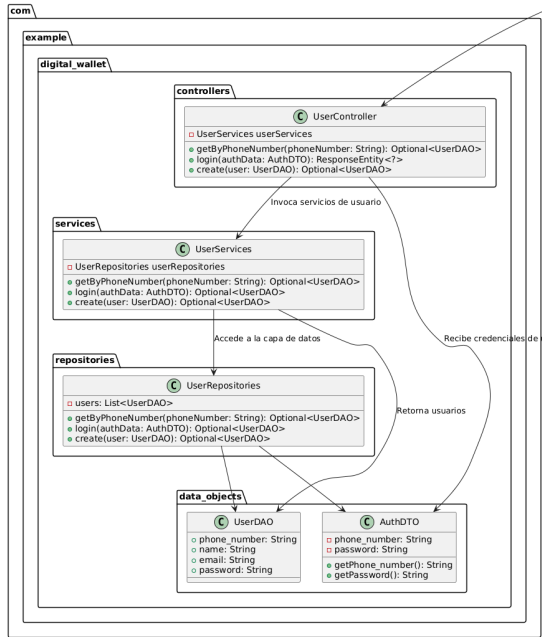


Fig. 2. Class Diagram java services

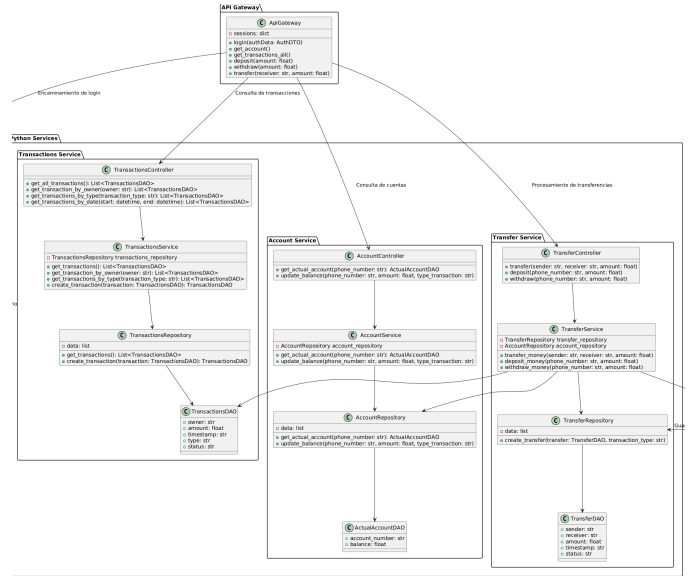


Fig. 3. Class Diagram API Gateway and Python services

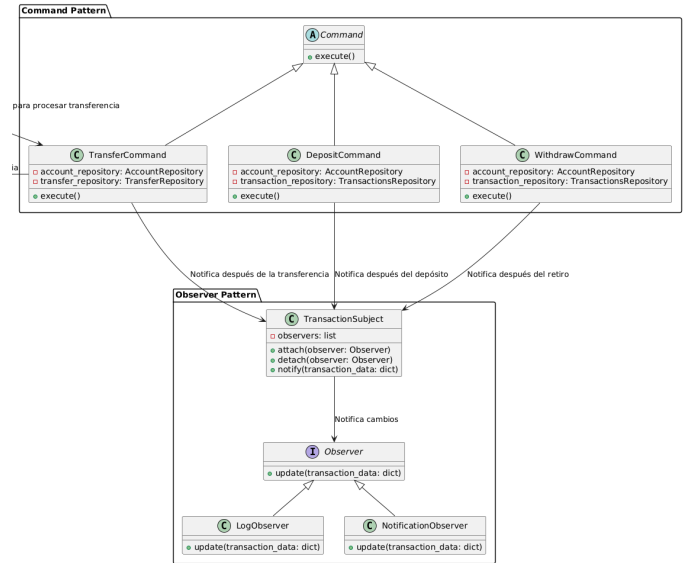


Fig. 4. class diagram command and observer pattern

A. Class Diagram

The class diagram represents the structure of the digital wallet system, showing the relationships between controllers, services, repositories, and data objects.

- **Controllers:** The system includes controllers such as *UserController*, *AccountController*, *TransactionsController*, and *TransferController*, which handle user requests and interact with services.
- **Services:** Each controller communicates with a corresponding service (*UserServices*, *AccountService*, *TransactionsService*, and *TransferService*) that encapsulates business logic and interacts with repositories.
- **Repositories:** The repository layer (*UserRepositories*, *AccountRepository*, *TransactionsRepository*, and *TransferRepository*)

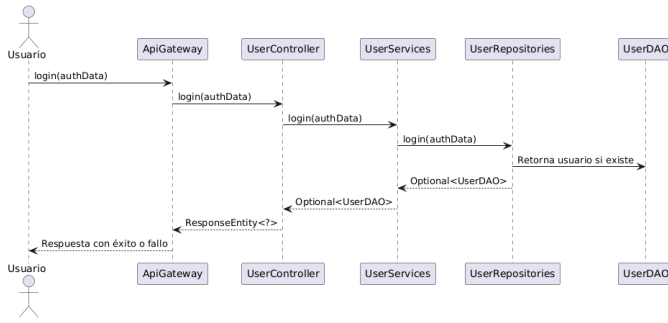


Fig. 5. Sequence Diagram of login

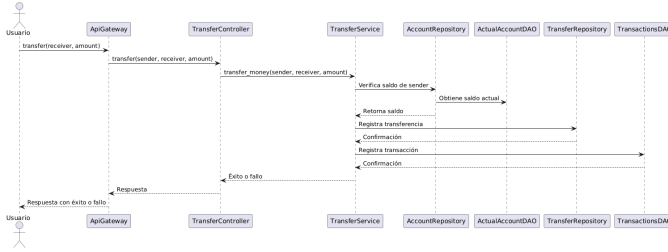


Fig. 6. sequence diagram of transfer money

TransferRepository) manages data storage and retrieval.

- **Data Objects:** The system includes entities such as *UserDAO*, *AuthDTO*, *ActualAccountDAO*, and *TransactionsDAO*, which represent data stored and manipulated within the application.
- **Patterns:** The system follows design patterns such as the *Command Pattern* for handling deposits, withdrawals, and transfers, and the *Observer Pattern* for transaction notifications.
- **API Gateway:** The *ApiGateway* component acts as an entry point, routing requests to appropriate services.

This class diagram provides a high-level overview of how the components interact and ensure separation of concerns, scalability, and maintainability in the digital wallet system.

B. Sequence diagram: Login and Money transfer

This diagram illustrates the login process of a user in the system. The user sends authentication data to the *ApiGateway*, which forwards the request to the *UserController*. The request is processed through the service and repository layers, checking for a valid user. Finally, the response is returned to the user, indicating whether the login was successful or not.

This diagram represents the process of transferring money between accounts. The user initiates the transfer through the *ApiGateway*, which communicates with the *TransferController*. The system verifies the sender's balance, updates account records, logs the transaction, and notifies observers before confirming the transfer result.

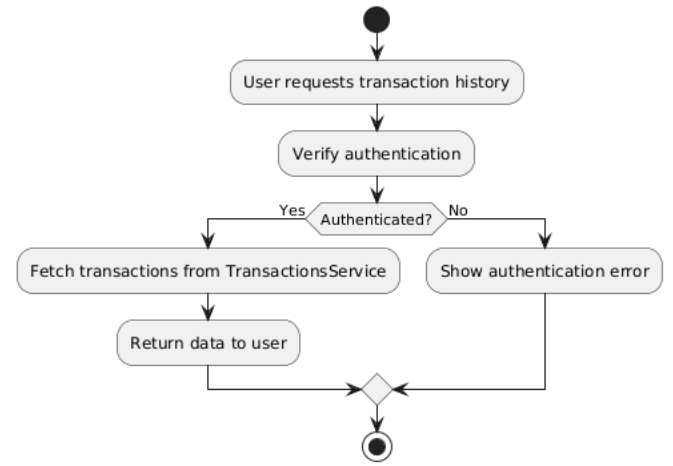


Fig. 7. Activity diagram of transaction history

C. Activity Diagram for transaction history and money transfer

This diagram outlines the transaction history retrieval process. The user requests transaction records, and the system verifies authentication. If authentication is valid, transaction data is fetched from the *TransactionsService* and displayed to the user. Otherwise, an authentication error is shown.

This diagram shows the steps involved in a money transfer operation. The process begins when the user initiates a transfer. If the sender has sufficient funds, the system registers the transfer, updates balances, and notifies observers. Otherwise, an error message is displayed to the user.

D. Deployment diagram

This diagram depicts the system's architecture and the distribution of services across different servers. It includes an *API Gateway*, a *User Service* and *Auth Service* hosted on a Java server, and the *Account Service*, *Transaction Service*, and *Transfer Service* running on a Python server.

E. Component diagram

This diagram describes the relationship between different components in the system. The *API Gateway* acts as an intermediary between the front-end and backend services. Each service, including user management, accounts, transactions, and transfers, consists of a controller, service, and repository layer for handling requests and data persistence.

V. SOLID PRINCIPLES IN ARCHITECTURE

- **Single Responsibility Principle (SRP)** The **Single Responsibility Principle (SRP)** states that each class or module should have only one reason to change, meaning that every component should focus on a single functionality. In the digital wallet project, SRP is applied through a microservices architecture, ensuring a clear separation of concerns between different system functionalities.

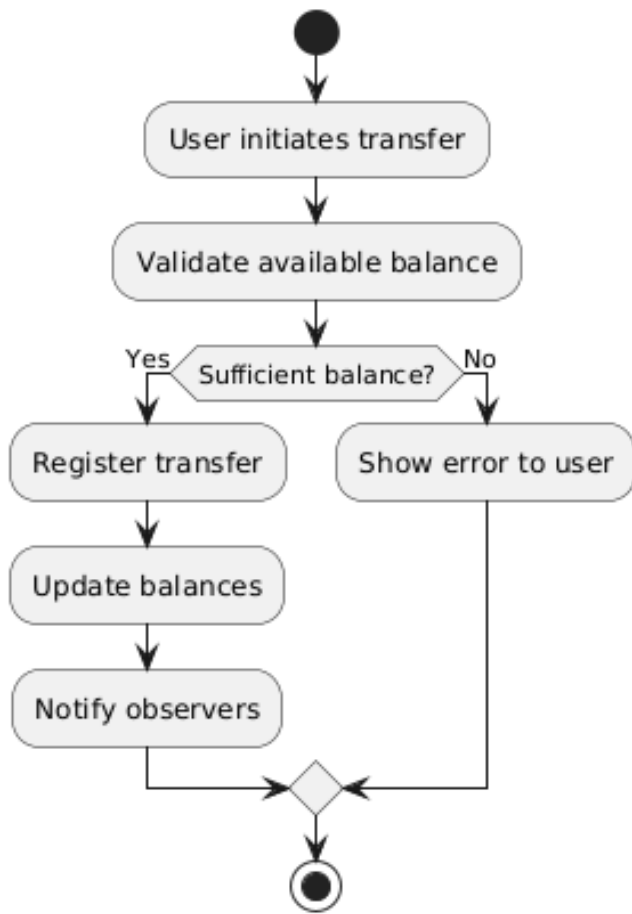


Fig. 8. activity diagram of money transfer

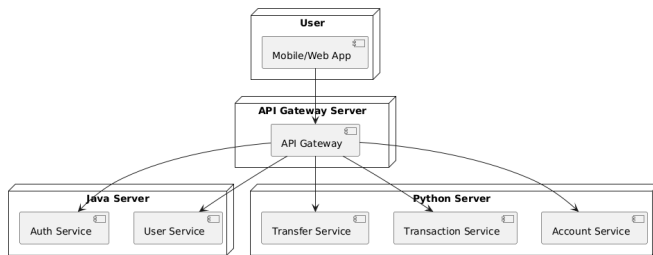


Fig. 9. Deployment diagram

The project consists of two main microservices: the **User Microservice**, implemented in Java using Spring Boot, and the **Transaction Microservice**, developed in Python. Each microservice follows a well-structured design pattern, where responsibilities are divided into repositories, services, and controllers. The **repositories** handle data persistence, ensuring that database operations are encapsulated. The **services** manage business logic, processing requests independently of data storage. Finally, the **controllers** are responsible for handling HTTP requests, ensuring that communication with clients is separated from core business operations.

In the **User Microservice**, SRP is enforced by keeping

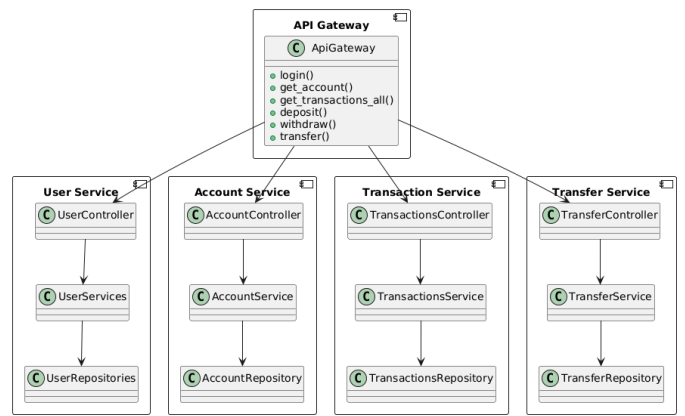


Fig. 10. Component diagram

authentication and user management responsibilities isolated from financial transactions. User authentication, login, and session management are handled within the user service, ensuring security concerns are not mixed with transaction processing. Similarly, the **Transaction Microservice** is responsible solely for handling deposits, withdrawals, and money transfers, without being concerned with user authentication. This clear separation ensures that modifications to one functionality do not impact unrelated components.

Additionally, the project maintains SRP at a higher level by separating concerns across different modules. A dedicated **notification system** ensures that real-time alerts for transactions are handled independently, preventing direct coupling with transaction processing. Likewise, a distinct **persistence layer** ensures that data management is modular, allowing potential future upgrades such as migrating from JSON storage to a structured database like PostgreSQL or MongoDB without affecting other system components.

By adhering to SRP, the digital wallet system improves maintainability, scalability, and modularity. Each component can evolve independently, allowing seamless updates and minimizing the risk of unintended side effects. Furthermore, this design enhances code reusability, as individual modules, such as authentication or transaction processing, can be reused or extended without requiring modifications to other parts of the system. Moving forward, this modular approach will facilitate the integration of additional features while maintaining a well-structured and maintainable code-base.

- The **Open-Closed Principle (OCP)** states that software entities (such as classes, modules, and functions) should be **open for extension but closed for modification**. This principle aims to ensure that new functionalities can be added without altering existing code, reducing the risk of introducing bugs.

Application in the Project

In this project, the Open-Closed Principle is applied in several ways, primarily through the use of the **Command Pattern** and the **Observer Pattern**.

Command Pattern: The Command interface

allows the system to introduce new commands (`DepositCommand`, `WithdrawCommand`, `TransferCommand`) without modifying existing functionality. Instead of changing the core transaction logic, new operations can be implemented as additional command classes that implement the `execute()` method.

- **Open for extension:** New commands can be added (e.g., `LoanRepaymentCommand`) without modifying existing command classes.
- **Closed for modification:** The `TransferService` class does not need changes when a new command is introduced; it simply executes the new command.

Observer Pattern: The Observer Pattern ensures that new observers (e.g., logging, notifications) can be introduced without modifying the core transaction logic. The `TransactionSubject` class handles observer management, allowing additional observers to be attached or detached dynamically.

- **Open for extension:** New observers (e.g., fraud detection, analytics) can be added without modifying existing ones.
- **Closed for modification:** The transaction execution flow does not need to be altered when adding new observers.

Benefits in the Project: By following the Open-Closed Principle:

- The code remains flexible and adaptable to new requirements.
- Existing functionality is preserved, reducing the risk of introducing errors.
- The system becomes more modular, making testing and debugging easier.

This principle significantly improves maintainability, making the project more scalable and robust over time.

- The **Liskov Substitution Principle (LSP)** states that objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program. This principle is fundamental for maintaining proper inheritance and ensuring that derived classes extend functionality without altering expected behavior.

In this project, LSP is applied in several ways. The **Command Pattern** ensures that all command classes, such as `DepositCommand`, `WithdrawCommand`, and `TransferCommand`, inherit from the `Command` interface and correctly implement the `execute()` method. This guarantees that any command can be used interchangeably within `TransferService` without modifying its logic. Similarly, the **Observer Pattern** follows LSP by allowing different observers, such as `LogObserver` and `NotificationObserver`, to be substituted seamlessly without requiring changes in the `TransactionSubject` class, as all observers adhere to the same `Observer` interface.

The project also applies LSP through its reliance on interfaces and abstract classes. For example, the `AccountService` depends on the

`AccountRepository`, which can be replaced with a different repository implementation, such as a database-driven repository instead of a JSON-based one, without affecting the service's core functionality. Additionally, the **API Gateway** exemplifies LSP by abstracting interactions with multiple services, ensuring that whether requests are routed to `UserService`, `TransactionService`, or `TransferService`, the responses remain consistent and predictable. This design allows for seamless substitution of service implementations without modifying the API Gateway itself.

Overall, the project correctly applies the Liskov Substitution Principle by ensuring that subclasses do not override behavior in a way that breaks expectations. Every class respects the contract defined by its parent class or interface, allowing polymorphism to function correctly. This adherence to LSP contributes to a flexible, maintainable, and scalable system where new features can be added without compromising existing functionality.

- The **Interface Segregation Principle (ISP)** states that a class should not be forced to implement methods it does not use. Instead of large, monolithic interfaces, systems should define smaller, more specific interfaces tailored to the exact needs of the implementing classes. This principle is essential for maintaining modularity and avoiding unnecessary dependencies.

In this project, ISP is applied effectively through the use of clearly defined interfaces and abstraction layers. The **Command Pattern** follows ISP by defining a minimal `Command` interface, which only requires the implementation of the `execute()` method. Each command class, such as `DepositCommand`, `WithdrawCommand`, and `TransferCommand`, implements only this method, avoiding unnecessary functionality that would not be relevant to all commands. Similarly, the **Observer Pattern** enforces ISP by defining a simple `Observer` interface with the `update()` method. This ensures that all observers, such as `LogObserver` and `NotificationObserver`, adhere to a single responsibility without being forced to implement unrelated behavior.

The project also applies ISP in its service and repository layers. The `AccountRepository`, `TransactionsRepository`, and `TransferRepository` each provide only the necessary methods for their respective entities, rather than a large, generic repository interface that would force all repositories to implement methods they do not need. This separation ensures that each repository is focused on its specific domain without unnecessary dependencies.

Furthermore, the **API Gateway** aligns with ISP by providing specialized endpoints for different functionalities. Instead of exposing a single, general-purpose API, it segregates responsibilities by handling authentication, account management, and transactions separately. This prevents the gateway from requiring services to conform to a broad and unfocused inter-

face.

By following the Interface Segregation Principle, the project maintains a clean and modular design, ensuring that classes and interfaces remain focused on their specific responsibilities. This improves maintainability, reduces coupling, and allows for easier modifications and extensions without affecting unrelated parts of the system.

- The **Dependency Inversion Principle (DIP)** states that high-level modules should not depend on low-level modules; instead, both should depend on abstractions. This principle ensures that the system remains flexible, reducing coupling between components and allowing for easier modifications and scalability. In this project, DIP is applied through the use of abstraction layers, particularly in the ****Command Pattern****, ****Observer Pattern****, and ****service-repository structure****. The ****Command Pattern**** adheres to DIP by ensuring that high-level components, such as `TransferService`, do not directly depend on specific transaction implementations. Instead of calling deposit, withdraw, or transfer logic directly, `TransferService` relies on the abstract `Command` interface, allowing for the seamless addition of new commands without modifying the service itself. Similarly, the ****Observer Pattern**** applies DIP by decoupling transaction handling from notification and logging mechanisms. Instead of `TransferService` or `TransactionsService` directly invoking logging or notification functions, they interact with the `TransactionSubject`, which relies on the `Observer` abstraction. This allows new observers, such as fraud detection or analytics, to be added without modifying the core transaction services. The project also follows DIP in the ****repository and service structure****, where each service depends on an abstract repository interface rather than a concrete implementation. For example, `AccountService`, `TransactionsService`, and `TransferService` do not interact directly with low-level storage implementations but instead depend on their respective repository abstractions (`AccountRepository`, `TransactionsRepository`, and `TransferRepository`). This abstraction allows for easy replacement of storage mechanisms, such as switching from JSON-based storage to a database, without modifying business logic. Furthermore, the ****API Gateway**** aligns with DIP by abstracting service interactions. Instead of directly depending on the internal implementation details of the `UserService`, `TransactionsService`, or `TransferService`, the API Gateway acts as an intermediary that communicates with services through well-defined APIs, making it easy to replace or update individual services without affecting others. By applying the Dependency Inversion Principle, the project ensures that modules remain loosely coupled, allowing for easy scalability and maintainability. Changes to low-level components, such as repositories, transaction handlers, or notification mechanisms,

do not require modifications in high-level modules, preserving the stability and extensibility of the system.

VI. ANALYSIS AND DETAIL OF THE IMPLEMENTATION OF DESIGN PATTERNS

The implementation of design patterns in this project plays a crucial role in ensuring maintainability, scalability, and flexibility. By leveraging well-established software design principles, the system achieves a modular structure that allows for easier modifications and future expansions. Several key design patterns were used, each addressing specific challenges within the architecture.

The **Command Pattern** was implemented to encapsulate transaction-related operations such as deposits, withdrawals, and transfers. Instead of directly embedding these operations into the `TransferService`, individual command classes (`DepositCommand`, `WithdrawCommand`, `TransferCommand`) were introduced. This approach ensures that the transaction handling logic remains decoupled from the service layer, making it easier to extend functionality without modifying existing code. The Command Pattern was an effective solution as it allows new transaction types to be introduced without altering the core business logic, adhering to the Open-Closed Principle.

The **Observer Pattern** was applied to decouple transaction execution from logging and notification mechanisms. Instead of embedding logging and notification logic directly within transaction services, the system defines a `TransactionSubject` class that manages observers such as `LogObserver` and `NotificationObserver`. When a transaction occurs, all registered observers are notified automatically. This design enables additional observers to be introduced, such as fraud detection or analytics services, without modifying the transaction execution logic. The implementation of the Observer Pattern proved effective in enhancing modularity and reducing tight coupling between components.

The **Repository Pattern** was used to abstract data access logic from business logic. Each service (`AccountService`, `TransactionsService`, and `TransferService`) interacts with its respective repository (`AccountRepository`, `TransactionsRepository`, and `TransferRepository`), ensuring that storage mechanisms remain interchangeable. This abstraction allows for the potential migration from JSON-based storage to a relational or NoSQL database with minimal code changes. The Repository Pattern effectively enhances maintainability by separating concerns and providing a clear data access layer.

The **API Gateway Pattern** was incorporated to centralize communication between clients and microservices. Instead of exposing multiple endpoints for authentication, transactions, and account management, the API Gateway provides a single entry point for external requests. This design improves security, simplifies client interactions, and reduces service dependencies by acting as an intermediary. The API Gateway Pattern has proven effective in streamlining request management and enabling service scalability.

Overall, the use of these design patterns has significantly improved the project's structure. The Command Pattern pro-

vided flexibility in transaction handling, the Observer Pattern ensured modularity in event-driven operations, the Repository Pattern maintained clean separation between business logic and data access, and the API Gateway Pattern enhanced service orchestration. The adoption of these patterns successfully addressed architectural challenges, making the system more adaptable, extensible, and maintainable.

VII ANALYSIS OF THE IMPLEMENTATION OF GOOD PROGRAMMING PRACTICES AND ANTI-PATTERNS.

The implementation of this project follows several programming best practices to ensure code quality, maintainability, and scalability. Adhering to structured software development principles has resulted in a modular, loosely coupled, and extensible system. However, while many best practices have been implemented, certain areas could be susceptible to anti-patterns, which may introduce potential issues if not properly managed.

One of the key best practices applied in this project is the use of **separation of concerns**, where distinct responsibilities are assigned to different components. The system is designed with a clear division between controllers, services, and repositories, ensuring that each layer focuses on a single aspect of the application's functionality. The use of the `AccountService`, `TransactionsService`, and `TransferService` prevents business logic from being mixed with data access operations, improving maintainability and testability.

Another best practice applied is **dependency injection**, particularly in the Java-based services, where dependencies such as repositories are injected rather than instantiated directly within the classes. This approach enhances modularity and facilitates testing by allowing mock dependencies to be used. In Python, similar principles are followed through class composition and clear interface design, ensuring that services interact with abstractions rather than concrete implementations.

Code reuse and modularity are also well-executed through the use of design patterns such as the Command Pattern and the Observer Pattern. The `Command` interface ensures that transaction-related logic is encapsulated in reusable and independent command classes, while the Observer Pattern allows for extensible event-driven functionality. These patterns prevent code duplication and promote the Open-Closed Principle, enabling new functionality to be added without modifying existing code.

Despite these best practices, certain anti-patterns could emerge if not carefully managed. One potential risk is the **God Object** anti-pattern, where a single class accumulates too many responsibilities. While the project successfully segregates functionality across multiple classes, excessive responsibility within the API Gateway could make it a central point of failure if it becomes too complex. Ensuring that the gateway remains lightweight and only acts as a request handler, without excessive business logic, is crucial to preventing this issue.

Another anti-pattern that could arise is **Shotgun Surgery**, where small changes in one part of the system require modifications across multiple components. While the use of interfaces and dependency injection reduces this risk, certain tightly

coupled dependencies, such as the repository implementations, could lead to unintended cascading changes. A potential improvement would be to further abstract data persistence layers to ensure minimal impact when switching storage backends.

Additionally, the project effectively avoids the **Spaghetti Code** anti-pattern by maintaining a structured and well-documented codebase. The adherence to naming conventions, meaningful method signatures, and modularized logic prevents complex interdependencies that would otherwise reduce code readability. Similarly, the use of logging mechanisms in the Observer Pattern ensures that debugging and monitoring can be performed without scattering print statements across the codebase.

Overall, the implementation of best practices in this project has significantly enhanced maintainability and scalability. The use of separation of concerns, dependency injection, and design patterns has resulted in a well-structured and flexible system. However, potential anti-patterns such as excessive reliance on the API Gateway or tightly coupled dependencies should be monitored to prevent long-term maintainability issues. By continuously refactoring and improving modularity, the project can further strengthen its adherence to best practices while minimizing risks associated with anti-patterns.

VIII CONCLUSION AND RECOMMENDATIONS

The implementation of design patterns and programming best practices in this project has significantly contributed to achieving a structured, maintainable, and scalable system. Through the application of the **Command Pattern**, **Observer Pattern**, **Repository Pattern**, and **API Gateway Pattern**, the system effectively separates concerns, promotes code reuse, and ensures that components remain loosely coupled. These design choices align with the SOLID principles, particularly the Open-Closed Principle and Dependency Inversion Principle, allowing for future extensions without modifying existing functionality.

The integration of programming best practices, such as dependency injection, modularity, and structured code organization, further enhances system maintainability. The project successfully avoids common anti-patterns such as Spaghetti Code and Shotgun Surgery, ensuring that modifications in one component do not require extensive changes across the system. However, certain risks, such as an overloaded API Gateway or potential tight coupling in repository implementations, should be monitored to prevent maintainability issues in the long term.

To summarize the impact of these architectural decisions, the following table presents an evaluation of key aspects of the implementation:

Aspect Evaluated	Effectiveness	Potential Issues
Command Pattern	High	None observed
Observer Pattern	High	Possible overuse in future extensions
Repository Pattern	High	Possible data persistence tight coupling
API Gateway	Moderate	Risk of excessive responsibility
Dependency Injection	High	Requires careful dependency management

TABLE I. EVALUATION OF DESIGN PATTERNS AND PRACTICES IN THE PROJECT

Based on these findings, several recommendations can be made for future improvements. First, it is advisable to ensure

that the **API Gateway remains lightweight** by offloading complex logic to microservices rather than centralizing too many responsibilities. Second, to further enhance scalability, **repository interfaces should be abstracted** to allow seamless switching between different storage backends, preventing potential tight coupling with a specific data persistence mechanism. Third, while the **Observer Pattern is effective**, its implementation should be carefully managed to prevent excessive event propagation that could lead to unnecessary processing overhead.

In conclusion, the current architecture effectively balances flexibility, scalability, and maintainability by leveraging appropriate design patterns and best practices. While the system is well-structured, continuous refactoring and monitoring of potential anti-patterns will be essential in maintaining its long-term efficiency. Future enhancements should focus on further modularization and optimizing the responsibility distribution across components to ensure the system remains adaptable to evolving requirements.